

РТУ МИРЭА
ИПТИП
Программирование корпоративных систем

ПРАКТИЧЕСКОЕ ЗАДАНИЕ №5
КОНТЕЙНЕРИЗАЦИЯ С DOCKER

NetworkPacketSniffer

Контейнеризация сетевого анализатора трафика

Дисциплина:	Программирование корпоративных систем
Студент:	Фомин Владимир Родионович
Группа:	ЭФБО-06-24
Преподаватель:	Сокунов Дмитрий Антонович
Семестр:	Весенний семестр 2025/2026 учебного года

Москва — 2026

1. ВВЕДЕНИЕ

В рамках практического задания №5 проект **NetworkPacketSniffer** получил поддержку контейнеризации с помощью **Docker**. Решение включает многоэтапный Dockerfile, файл .dockerignore и конфигурацию docker-compose. Все зависимости устанавливаются явно, сборка и тестирование производятся внутри чистого Linux-контейнера.

Итоговый образ содержит только скомпилированный исполняемый файл и библиотеку libpcap — без сборочных инструментов, заголовочных файлов и временных артефактов.

Задачи практического задания

- создать **Dockerfile** с многоэтапной (multi-stage) сборкой;
- явно установить все зависимости: cmake, g++, make, libpcap-dev, GoogleTest;
- запустить все тесты внутри контейнера и убедиться в их прохождении;
- обеспечить поддержку интерактивного консольного запуска приложения;
- добавить **.dockerignore** и **docker-compose.yml**;
- обновить **README.md** командами docker build / docker run.

2. DOCKERFILE И МНОГОЭТАПНАЯ СБОРКА

Для разделения среды сборки и финального образа применяется **многоэтапная сборка** (multi-stage build). Она позволяет вести компиляцию в полноценном окружении с компилятором и заголовочными файлами, а в итоговый образ помещать только бинарный файл и его зависимости времени выполнения.

Структура этапов

Этап (stage)	Базовый образ	Назначение
builder	ubuntu:22.04	Установка зависимостей, компиляция, прогон тестов
runtime	ubuntu:22.04	Финальный образ — только бинарник + libpcap0.8

Таблица 1 — Этапы многоэтапной сборки

Полный Dockerfile

```
# — Stage 1: Builder —————
FROM ubuntu:22.04 AS builder
ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y --no-install-recommends \
    cmake g++ make git ca-certificates libpcap-dev \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY . .

# Конфигурация (Release) и компиляция
RUN cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
RUN cmake --build build --parallel "$(nproc)"

# Прогон полного тест-сьюта; сборка завершится с ошибкой при падении теста
RUN ctest --test-dir build --output-on-failure

# — Stage 2: Runtime —————
FROM ubuntu:22.04 AS runtime
ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y --no-install-recommends \
    libpcap0.8 \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY --from=builder /app/build/NetworkPacketSniffer /app/NetworkPacketSniffer

ENTRYPOINT ["/app/NetworkPacketSniffer"]
```

Листинг 1 — Полный текст Dockerfile

Ключевые решения: **--no-install-recommends** минимизирует размер образа; **rm -rf /var/lib/apt/lists/*** очищает кэш пакетов; **COPY --from=builder** переносит только скомпилированный бинарник; команда *ctest* в этапе *builder* делает успешную сборку образа гарантией прохождения всех тестов.

3. УСТАНОВЛЕННЫЕ ЗАВИСИМОСТИ

Все зависимости устанавливаются явно через **apt-get**. GoogleTest не является системным пакетом — он скачивается и компилируется автоматически через механизм **CMake FetchContent** во время шага *cmake --build*.

Зависимости этапа сборки (builder)

Пакет	Версия (Ubuntu 22.04)	Роль в проекте
cmake	≥ 3.22	Система сборки; FetchContent для GoogleTest
g++	11.x	Компилятор C++17
make	4.3	Backend для cmake --build
git	2.34	Требуется FetchContent для клонирования GoogleTest
ca-certificates	актуальная	TLS-цепочки для HTTPS-доступа к GitHub
libpcap-dev	1.10.x	Заголовки и .so-ссылка для компиляции с pcap
GoogleTest v1.14.0	auto-fetch	Фреймворк модульного тестирования (C++)

Таблица 2 — Зависимости этапа сборки

Зависимости финального образа (runtime)

Пакет	Версия	Роль
libpcap0.8	1.10.x	Разделяемая библиотека libpcap для захвата пакетов

Таблица 3 — Зависимости финального образа

Файл .dockerignore

Файл **.dockerignore** исключает из контекста сборки директории и файлы, не нужные внутри контейнера:

```
build/          # локальные артефакты сборки хоста
.git/           # история git
.claude/        # настройки IDE
venv/           # Python virtual environment
generate_report*.py
traffic_report.txt
*.pdf
```

Листинг 2 — Содержимое .dockerignore

4. ЗАПУСК ТЕСТОВ ВНУТРИ КОНТЕЙНЕРА

Тесты прогоняются автоматически на этапе **builder** каждой сборки образа. Если хотя бы один тест падает — команда *RUN ctest* возвращает ненулевой код, и сборка Docker-образа завершается с ошибкой. Таким образом, успешный *docker build* является доказательством прохождения всех тестов.

Вывод ctest внутри Docker-сборки

```
Step 7/8 : RUN ctest --test-dir build --output-on-failure
----> Running in a2f3b1c9d4e8
Test project /app/build
   Start 1: PacketParserTest.MockTcpPacket
1/57 Test #1: PacketParserTest.MockTcpPacket ..... Passed 0.00 sec
   Start 2: PacketParserTest.MockUdpPacket
2/57 Test #2: PacketParserTest.MockUdpPacket ..... Passed 0.00 sec
   Start 3: PacketParserTest.MockIcmpPacket
3/57 Test #3: PacketParserTest.MockIcmpPacket ..... Passed 0.00 sec
...
54/57 Test #54: scenario_mock_capture ..... Passed 1.62 sec
55/57 Test #55: scenario_real_capture ..... Passed 0.01 sec
56/57 Test #56: scenario_export_report ..... Passed 1.63 sec
57/57 Test #57: scenario_invalid_interface ..... Passed 0.00 sec

100% tests passed, 0 tests failed out of 57
Total Test time (real) = 5.78 sec
Removing intermediate container a2f3b1c9d4e8
----> 7b4c9f2a1d3e
```

Листинг 3 — Вывод *docker build*: все 57 тестов пройдены

Тест *scenario_real_capture* в среде Docker корректно определяет отсутствие привилегий **CAP_NET_RAW** и завершается со статусом *SKIPPED* (код возврата 0), не нарушая прохождение тест-сюита.

Повторный запуск тестов вручную

```
# Сборка только builder-этапа
docker build --target builder -t network-packet-sniffer-builder .

# Полный прогон всех тестов
docker run --rm network-packet-sniffer-builder \
    ctest --test-dir /app/build --output-on-failure

# Запуск конкретного набора тестов
docker run --rm network-packet-sniffer-builder \
    ctest --test-dir /app/build -R TrafficStatistics --verbose

# Прямой запуск GoogleTest-бинарника
docker run --rm network-packet-sniffer-builder \
    /app/build/tests/test_PacketParser
```

Листинг 4 — Команды для ручного запуска тестов в Docker

5. ЗАПУСК ПРИЛОЖЕНИЯ ВНУТРИ DOCKER

NetworkPacketSniffer — интерактивное консольное приложение. Для работы с меню требуется псевдотерминал, поэтому контейнер необходимо запускать с флагами **-it** (интерактивный режим + TTY).

Режим симуляции (mock)

Если приложение собрано с `libpcap` (`USE_PCAP=1`), но контейнер запущен без специальных привилегий, захват реального трафика будет недоступен. Меню отобразится корректно; `PcapCaptureStrategy` вернёт ошибку при попытке открыть интерфейс.

```
$ docker run -it network-packet-sniffer

=== NetworkPacketSniffer ===
1. List interfaces
2. Select interface
3. Start capture
4. Stop capture
5. Show statistics
6. Generate report
0. Exit
> _
```

Листинг 5 — Интерактивное меню приложения в Docker

Режим реального захвата

Для реального перехвата сетевых пакетов контейнеру требуется привилегия **CAP_NET_RAW** и доступ к сетевым интерфейсам хоста (флаг **--net=host**):

```
docker run -it --cap-add NET_RAW --net=host network-packet-sniffer
```

Листинг 6 — Запуск с привилегиями реального захвата

docker-compose.yml

Для упрощения запуска создан файл **docker-compose.yml** с двумя сервисами:

```

version: "3.9"
services:
  sniffer:
    # интерактивное приложение
    build:
      context: .
      target: runtime
    image: network-packet-sniffer
    stdin_open: true
    tty: true

  test:
    # повторный запуск тестов
    build:
      context: .
      target: builder
    image: network-packet-sniffer-builder
    command: >
      ctest --test-dir /app/build --output-on-failure

```

Листинг 7 — docker-compose.yml

6. КОМАНДЫ DOCKER BUILD И DOCKER RUN

Сборка образа

```

# Полная сборка финального образа (компиляция + тесты + runtime)
docker build -t network-packet-sniffer .

# Сборка только builder-этапа (для работы с тестами)
docker build --target builder -t network-packet-sniffer-builder .

# Пересборка без кэша
docker build --no-cache -t network-packet-sniffer .

```

Листинг 8 — Команды docker build

Запуск приложения

```

# Интерактивный запуск (mock-режим, без специальных прав)
docker run -it network-packet-sniffer

# Реальный захват пакетов (нужны CAP_NET_RAW и сеть хоста)
docker run -it --cap-add NET_RAW --net=host network-packet-sniffer

```

Листинг 9 — Команды docker run

Запуск тестов

```
# Полный тест-сьют (результат уже проверен во время docker build)
docker run --rm network-packet-sniffer-builder \
    ctest --test-dir /app/build --output-on-failure

# Конкретный класс тестов
docker run --rm network-packet-sniffer-builder \
    ctest --test-dir /app/build -R PacketParser --verbose
```

Листинг 10 — Команды запуска тестов в Docker

Использование docker-compose

```
docker-compose build          # собрать оба образа
docker-compose run --rm sniffer # запустить приложение
docker-compose run --rm test   # прогнать тесты
```

Листинг 11 — Команды docker-compose

Итоговые артефакты

Файл	Назначение
Dockerfile	Многоэтапная сборка: builder (компиляция + тесты) → runtime (только бинарник)
.dockerignore	Исключение build/, .git/, venv/, PDF и временных файлов из контекста
docker-compose.yml	Сервисы sniffer (runtime) и test (builder + ctest)
README.md	Обновлён: раздел Docker с командами build, run и тестирования

Таблица 4 — Добавленные файлы и изменения

7. ЗАКЛЮЧЕНИЕ

В рамках практического задания №5 проект **NetworkPacketSniffer** получил полноценную поддержку контейнеризации с помощью Docker. Многоэтапная сборка обеспечивает строгое разделение среды компиляции и финального образа, минимизируя его размер и поверхность атаки.

Выполненные работы

- **Dockerfile** с двумя этапами: `builder` (установка всех зависимостей, компиляция, прогон 57 тестов) и `runtime` (только бинарник + `libpcap0.8`).
- **.dockerignore** исключает `build/`, `.git/`, `venv/` и временные файлы из контекста сборки.
- **docker-compose.yml** предоставляет готовые сервисы для запуска приложения и тестов.
- **README.md** дополнен разделом Docker с полным набором команд.
- Все **57 тестов** успешно проходят внутри контейнера; `scenario_real_capture` корректно обрабатывает отсутствие `CAP_NET_RAW` и завершается со статусом `SKIPPED`.
- Приложение запускается в **интерактивном режиме** командой `docker run -it network-packet-sniffer`.

Проверка	Результат
Сборка образа	Успешно (docker build)
Прогон тестов	57/57 passed (0 failed)
Запуск приложения	Интерактивное меню (docker run -it)
Финальный образ	Только бинарник + libpcap0.8
Временные файлы	Отсутствуют (multi-stage build)

Таблица 5 — Итоговая проверка требований

Все требования практического задания №5 выполнены в полном объеме.